

Generative Adaptation of Temporal Point Processes for Generalizable Event Prediction and Sequence Simulation

Anonymous submission

Abstract

Real-world event sequences, e.g. urban transportation and on-line shopping behaviors, often have time-varying generative mechanisms driven by various contexts or conditions. As a result, learning a single static Temporal Point Process (TPP) model suffers from severe generalizability issues, limiting its practical feasibility. To overcome this issue, we propose a generative adaptation method for TPPs that generates model parameters based on sequence conditions and enhances model generalizability accordingly. In principle, given a pre-trained TPP model, we adapt it to grouped training event sequences via learning a set of Low-Rank Adapters (LoRAs) and then train a diffusion model to generate these adapters conditioned on various sequence-level side information. In inference, given new event sequences with unseen side information, we apply the diffusion model to generate adapters for TPP, thereby enabling it to generalize to out-of-distribution scenarios. Experiments across diverse real-world domains show that it significantly outperforms state-of-the-art baselines in both event prediction and sequence simulation under challenging scenarios, validating the effectiveness of generative adaptation for generalizable events modeling.

Introduction

Event sequences are discrete events occurring in continuous time. Modeling such data plays a central role in many application scenarios, e.g., urban transportation (Kalair, Connaughton, and Alaimo Di Loro 2021), online shopping behaviors (Sun et al. 2024), social network analysis (Zhou, Zha, and Song 2013), and patient admission modeling (Xu et al. 2016). Temporal Point Processes (TPPs) (Kingman 1992) and their neural variants (Mei and Eisner 2017; Yang, Mei, and Eisner 2021) characterize the instantaneous event occurrence rate, and have become dominant tools for event sequence prediction and simulation (Xue et al. 2023).

However, existing TPPs often suffer from limited generalizability in practice because real-world event sequences have time-varying generative mechanisms driven by various contexts or conditions. For example, urban taxi demand varies across day type, location, and weather. It shows rush-

hour peaks on weekdays but dispersed activity on weekends. Similarly, online shopping behaviors shift with user profiles and promotional seasons. Such condition-dependent dynamics lead to inevitable distribution shifts between training and deployment, which cannot be captured by a single static TPP model. Existing methods mitigate this issue either through input-level conditioning (Okawa et al. 2019; Kong et al. 2026), which feeds condition information into a single shared model, or through parameter-efficient adaptation such as LoRA (Liu and Quan 2024), which fine-tunes a small set of weights. The former reuses one parameter set across all conditions with only coarse-grained control, while the latter needs per-condition data and gradient updates for each new condition. Both yield coarse-grained adaptation and suboptimal out-of-distribution performance.

To overcome this issue, we propose **Generative Adaptation of Temporal Point Processes (GA-TPP)**, which generates model parameters based on sequence-level conditions and enhances model generalizability effectively. As illustrated in Figure 1, given event sequences with clustered context (e.g., aggregated data statistics and textual environmental descriptions), we pretrain a TPP model based on event sequences and then fine-tune it to derive a set of cluster-specific Low-Rank Adapters (LoRAs) (Hu et al. 2021). We then train a diffusion model (Ho, Jain, and Abbeel 2020) to generate these adapters conditioned on various context. During inference, given new context, the diffusion model generates a condition-specific adapter and injects it into the pretrained TPP, enabling the model to generalize to out-of-distribution scenarios.

To our knowledge, our work is the first attempt to achieve dynamic TPP modeling through generative adaptation, i.e., treating TPP parameters as the target of conditional generation. Because adaptation is determined entirely by sequence-level conditions, a single pretrained TPP with one learned diffusion prior can serve diverse conditions, requiring only adapter generation and injection at inference. Experiments across real-world domains demonstrate that GA-TPP significantly outperforms state-of-the-art baselines on both event prediction and sequence simulation, under in-distribution as well as challenging out-of-distribution scenarios.

Related Work

As aforementioned, temporal point processes have been widely used for continuous-time event sequence modeling in

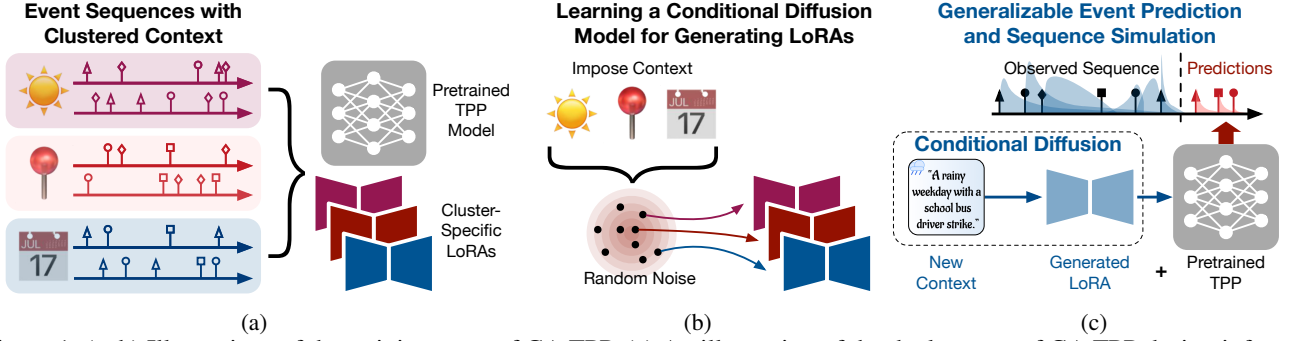


Figure 1: (a, b) Illustrations of the training steps of GA-TPP. (c) An illustration of the deployment of GA-TPP during inference.

real-world applications (Kalair, Connaughton, and Alaimo Di Loro 2021; Sun et al. 2024; Zhou, Zha, and Song 2013). Early TPPs, such as Poisson processes (Kingman 1992) and Hawkes processes (Hawkes 1971), rely on manually designed parametric forms, limiting their capacity for complex event sequences. Neural TPPs address this limitation with the help of deep neural networks, especially recurrent architectures (Du et al. 2016; Mei and Eisner 2017) and Transformer-based models (Zuo et al. 2020; Zhang et al. 2020; Yang, Mei, and Eisner 2021), enabling flexible modeling of long-range dependencies and heterogeneous event types. Recent benchmarks (Xue et al. 2023) and surveys (Shchur et al. 2021) demonstrate that neural TPPs are core tools for event sequence prediction and generation.

Existing methods commonly learn TPPs to fit training sequences via Maximum Likelihood Estimation (MLE) (Kingman 1992). To improve generalizability, many efforts have been made, e.g., imposing regularizers (Zhou, Zha, and Song 2013; Zhang, Lipani, and Yilmaz 2021), augmenting data (Xu, Luo, and Zha 2017), and adopting alternative learning paradigms (such as contrastive (Mei, Wan, and Eisner 2020; Wang et al. 2023), adversarial (Xiao et al. 2017), and reinforcement learning (Li et al. 2018)). However, all these methods learn a single static TPP to fit a presumed stationary process, whose performance degrades when data distribution drifts. Recently, meta-learning enables condition-specific model adaptation (Bae et al. 2023), but it still requires gradient-based fine-tuning for each new condition, whose scalability is questionable in practice.

Achieving generalizable TPPs calls for adaptation mechanisms beyond fitting a static model. Originating from controllable image generation (Zhang, Rao, and Agrawala 2023; Mou et al. 2024), a promising strategy is to modulate a pre-trained model with lightweight conditioning modules rather than retraining it. Recent methods push this idea further by directly generating model parameters from task or context descriptors, via hypernetworks (Ha, Dai, and Le 2016) or diffusion priors (Shen et al. 2025), achieving strong generalization to unseen attributes in recommendation tasks. Within the TPP literature, diffusion models have so far been used to model the event data itself. The spatio-temporal diffusion point processes in (Yuan et al. 2023) capture complex spatio-temporal dependencies in event data, while the conditional generative model in (Dong, Fan, and Zhu 2025) lever-

ages high-dimensional contexts for controllable simulation of richly marked sequences. These methods impose context descriptors at the input level, achieving controllable adaptation of pretrained TPPs. However, none of them considers generating model parameters for TPPs. Our work fills this gap by learning a diffusion prior over LoRA parameters, allowing a single pretrained TPP to adapt to unseen conditions without gradient updates.

Proposed Method

Suppose that we have a set of event sequences, denoted as $\mathcal{D} = \{\mathbf{s}_n\}_{n=1}^N$. Here, each sequence $\mathbf{s}_n = \{(t_j^n, c_j^n)\}_{j=1}^{J_n}$ consists of J_n events, and for each event, its timestamp $t_j^n \in [0, T_n]$ and types $c_j^n \in \mathcal{C} = \{1, \dots, C\}$. Furthermore, the event sequences can belong to M clusters, and each cluster is indicated by the sequence-level contexts that affect event dynamics significantly. For simplicity, we denote each context by $\mathbf{e}_m \in \mathbb{R}^D$, the D -dimensional embedding achieved by a predefined/pretrained model (e.g., an LLM (Zhang et al. 2025)). According to the clustered contexts, we can represent $\mathcal{D}$ as M subsets, i.e., $\{\mathcal{D}_m\}_{m=1}^M$.

As illustrated in Figure 1, the proposed GA-TPP method learns a diffusion-based generative prior over the adapters of a pretrained TPP model. During inference, this prior generates model adapters conditioned on different contexts, including out-of-distribution ones. To achieve this aim, GA-TPP learns the model by the following two steps.

Pretraining and Adaptation of TPP

Typically, a TPP model characterizes event dynamics by the conditional intensity functions, denoted as $\lambda(t; \theta) = \{\lambda_c(t; \theta)\}_{c \in \mathcal{C}}$, where $\lambda_c(t; \theta)$ quantifies the expected instantaneous rate of a type- c event at time t given historical events:

$$\lambda_c(t; \theta) = d\mathbb{E}[N_c(t; \theta) | \mathcal{H}_t]/dt, \quad \forall c \in \mathcal{C}, \quad (1)$$

where θ denotes the model parameters, $\mathcal{H}_t = \{(t_k, c_k) \in \mathcal{s} \mid t_k < t\}$, and $N_c(t; \theta)$ is the counting process for type- c events up to time t .

Following existing works, we can parameterize $\lambda_c(t; \theta)$ by various neural networks (Du et al. 2016; Mei and Eisner 2017; Zhang et al. 2020; Zuo et al. 2020; Yang, Mei, and Eisner 2021) and learn the model parameters accordingly. Regardless of the specific architecture, we first pretrain a

neural TPP in the MLE framework, based on all observed event sequences:

$$\max_{\theta} \sum_{s_n \in \mathcal{D}} \left[\sum_{j=1}^{J_n} \log \lambda_{c_j^n}(t_j^n; \theta) - \sum_{c \in \mathcal{C}} \int_0^{T_n} \lambda_c(\tau; \theta) d\tau \right], \quad (2)$$

where the integral is approximated by Monte Carlo sampling, and the log-likelihood can be optimized with Adam.

Notably, in the pretraining phase, we ignore the clustered contexts to simplify the pretraining task and learn a single static TPP purely based on event sequences. The pretrained model, denoted as θ^* , captures the global data distribution coarsely, which provides a good initialization for downstream context-specific model adaptation.

In particular, given the pretrained TPP θ and a cluster $\mathcal{D}_m$, we learn a Low-Rank Adapter (LoRA) (Hu et al. 2021) on $\mathcal{D}_m$ and inject it into the model. Denote the parameters of the m -th LoRA module as Δ_m . Similar to the pretrained model, we learn the module in the MLE framework as well

$$\max_{\Delta_m} \sum_{s_n \in \mathcal{D}_m} \left[\sum_{j=1}^{J_n} \log \lambda_{c_j^n}(t_j^n; \theta^* + \Delta_m) - \sum_{c \in \mathcal{C}} \int_0^{T_n} \lambda_c(\tau; \theta^* + \Delta_m) d\tau \right], \quad (3)$$

where $\theta^* + \Delta_m$ represents the adapted model obtained by injecting the LoRA into the pretrained TPP.

Example. Suppose that we would like to pretrain and adapt a THP model (Zuo et al. 2020). Given an event sequence $s = \{(t_j, c_j)\}_{j=1}^J$, the model first embeds each event as $\mathbf{h}_j^{(0)} = \text{PE}(t_j) + \mathbf{c}_{c_j}$, where $\mathbf{c}_{c_j} \in \mathbb{R}^d$ is a trainable type embedding and $\text{PE}(\cdot)$ is sinusoidal positional encoding. The embeddings are then passed through L Transformer encoder layers (Vaswani 2017), yielding hidden states $\{\mathbf{h}_j^{(L)}\}_{j=1}^J$. Given the most recent event at t_j ,

$$\lambda_c(t; \theta) = \text{Softplus}(\beta_c(t - t_j) + \mathbf{w}_c^\top \mathbf{h}_j^{(L)} + \gamma_c), \quad (4)$$

where $\beta_c, \gamma_c \in \mathbb{R}$ are learnable decay and bias parameters, $\mathbf{w}_c \in \mathbb{R}^d$ is a projection vector, and $\text{Softplus}(x) = \log(1 + e^x)$ ensures non-negativity. After pretraining the THP via (2), we adapt the query and value projection matrices of its L Transformer layers, denoted as $\{\mathbf{W}_q^{(\ell)}, \mathbf{W}_v^{(\ell)} \in \mathbb{R}^{d \times d}\}_{\ell=1}^L$. Taking the value projection as an example: for $\ell = 1, \dots, L$,

$$\mathbf{W}_v^{(\ell, m)} \leftarrow \mathbf{W}_v^{(\ell)} + \frac{\alpha}{r} \mathbf{B}_m^{(\ell)} \mathbf{A}_m^{(\ell)}, \quad (5)$$

where $r \ll d$, $\mathbf{A}_m^{(\ell)} \in \mathbb{R}^{r \times d}$ and $\mathbf{B}_m^{(\ell)} \in \mathbb{R}^{d \times r}$ are trainable low-rank matrices for the m -th cluster, and $\alpha > 0$ is a scaling hyperparameter; $\mathbf{W}_q^{(\ell)}$ is adapted in the same way with its own low-rank pair. The choice of the adapted matrices and layers is validated in our ablation studies (Table 4). Accordingly, Δ_m collects all the low-rank matrices of the m -th cluster into a single vector via $\text{vec}(\cdot)$. When adapting a pretrained RNN-based TPP model (Mei and Eisner 2017), we learn and inject LoRAs into its LSTM gate weight matrices.

Remark. Prior approaches incorporate conditions in two ways. Input-level conditioning (Okawa et al. 2019; Kong et al. 2026) injects condition information into the event representations at the input level (e.g., via concatenation or context-driven modulation), leaving the model parameters shared across all conditions. Parameter-efficient adaptation instead fine-tunes a small set of parameters: TPP-LLM (Liu and Quan 2024), for instance, learns a single global LoRA to adapt a pretrained LLM to the TPP task. In both cases, however, the same set of (adapted) parameters is reused for every condition. In contrast, our LoRA-based generative adaptation produces a condition-specific adapter per cluster, directly modulating the model’s internal computations at the parameter level and enabling fine-grained control over the intensity function while preserving the pretrained knowledge.

Generative Adaptation via Conditional Diffusion

The above conditional adaptation step yields a set of M adapters paired with their context embeddings, denoted as $\mathcal{S} = \{(\Delta_m, \mathbf{e}_m)\}_{m=1}^M$. Based on this dataset, we train a diffusion model $p_\phi(\Delta|e)$ to generate adapters conditioned on the embeddings.

In the training stage, we adopt the standard Gaussian diffusion framework (Ho, Jain, and Abbeel 2020; Song et al. 2020). Given an adapter $\Delta \in \mathcal{S}$, the forward process gradually adds noise to it over K steps. At step k , the conditional distribution and the corresponding sampling step are

$$q(\Delta^{(k)} | \Delta) = \mathcal{N}(\Delta^{(k)}; \sqrt{\bar{\alpha}_k} \Delta, (1 - \bar{\alpha}_k) \mathbf{I}), \quad (6)$$

$$\Delta^{(k)} = \sqrt{\bar{\alpha}_k} \Delta + \sqrt{1 - \bar{\alpha}_k} \epsilon, \quad \text{with } \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

where $\bar{\alpha}_k = \prod_{s=1}^k \alpha_s$ is the cumulative noise schedule with $\bar{\alpha}_K \approx 0$. The reverse process is parameterized by a neural network $\epsilon_\phi(\Delta^{(k)}, k, e)$ that predicts the added noise conditioned on the embedding e : for $k = 1, \dots, K$,

$$p_\phi(\Delta^{(k-1)} | \Delta^{(k)}, e) = \mathcal{N}(\Delta^{(k-1)}; \mu_\phi(\Delta^{(k)}, k, e), \sigma_k^2 \mathbf{I})$$

with $\mu_\phi(\Delta^{(k)}, k, e) = \frac{1}{\sqrt{\alpha_k}} (\Delta^{(k)} - \frac{1 - \alpha_k}{\sqrt{1 - \bar{\alpha}_k}} \epsilon_\phi(\Delta^{(k)}, k, e))$, and $\sigma_k^2 = \frac{(1 - \bar{\alpha}_{k-1})(1 - \alpha_k)}{1 - \bar{\alpha}_k}$ follows the DDPM schedule (Ho, Jain, and Abbeel 2020).

Following standard diffusion training (Ho, Jain, and Abbeel 2020), we minimize the mean squared error between the predicted and actual noise:

$$\min_{\phi} \mathbb{E}_{(\Delta, e) \sim \mathcal{S}, k, \epsilon} \left[\|\epsilon - \epsilon_\phi(\Delta^{(k)}, k, e)\|_2^2 \right] \quad (7)$$

where $k \sim \mathcal{U}(1, K)$ and $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. This problem is equivalent to maximizing a variational lower bound on $\log p_\phi(\Delta|e)$ (Ho, Jain, and Abbeel 2020). Algorithm 1 summarizes the overall training scheme.

Inference with Generative Adaptation

Leveraging the learned diffusion model, denoted as ϕ^* , we can generate various adapters for unseen conditions, enhancing the generalizability of the target TPP in zero-shot scenarios. During inference, given a new condition, we first

Algorithm 1 The Training Scheme of GA-TPP

Require: Event sequences $\mathcal{D} = \{\mathbf{s}_j\}_{j=1}^N$ with side information, the number of clusters M

Ensure: A pretrained TPP θ^* and a diffusion model ϕ^*

- 1: Pretrain θ via solving (2).
 - 2: **for** $m = 1, \dots, M$ **do**
 - 3: Learn a LoRA Δ_m via solving (3).
 - 4: **end for**
 - 5: Train a conditional diffusion model ϕ on the adapter-context pairs via solving (7).
-

construct its condition embedding e by encoding textual descriptions and/or aggregating event features. We then draw an adapter with the standard DDPM ancestral sampler (Ho, Jain, and Abbeel 2020), running the full reverse process: Starting from $\Delta^{(K)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, for $k = K, \dots, 1$,

$$\Delta^{(k-1)} = \frac{1}{\sqrt{\alpha_k}} \left(\Delta^{(k)} - \frac{1 - \alpha_k}{\sqrt{1 - \bar{\alpha}_k}} \epsilon_\phi(\Delta^{(k)}, k, e) \right) + \sigma_k \xi_k, \quad (8)$$

where $\xi_k \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ for $k > 1$ and $\xi_1 = \mathbf{0}$, and σ_k follows the DDPM schedule. The final sample $\Delta \leftarrow \Delta^{(0)}$ is the generated context-specific adapter. Because adapters lie in a low-dimensional space, this sampling adds negligible overhead compared with gradient-based per-condition adaptation, which we quantify in the experimental part.

Injecting Δ into the pretrained TPP adapts the model to the corresponding condition, supporting both accurate event prediction and condition-consistent sequence simulation.

- **Event prediction:** Given the history $\mathcal{H}_{t_j}$ up to the last observed event t_j , the survival function of the next event time T_{next} is

$$P(T_{\text{next}} > t \mid \mathcal{H}_{t_j}; \theta^* + \Delta) = \exp \left(- \int_{t_j}^t \sum_{c \in \mathcal{C}} \lambda_c(\tau; \theta^* + \Delta) d\tau \right), \quad (9)$$

which quantifies the probability that no event occurs in $(t_j, t]$. The probability that the next event is of type c , given that it occurs at time t , is

$$P(c \mid t, \mathcal{H}_{t_n}; \theta^* + \Delta) = \frac{\lambda_c(t; \theta^* + \Delta)}{\sum_{c' \in \mathcal{C}} \lambda_{c'}(t; \theta^* + \Delta)}. \quad (10)$$

- **Sequence simulation:** Injecting the generated adapter Δ into the pretrained TPP, we simulate event sequences from the adapted intensity function via Ogata’s thinning algorithm. Specifically, at each step, we first sample a candidate time increment from an upper-bound intensity $\bar{\lambda}$:

$$\Delta t \sim \text{Exp}(\bar{\lambda}), \quad \text{with } \bar{\lambda} \geq \sum_{c \in \mathcal{C}} \lambda_c(t; \theta^* + \Delta), \quad (11)$$

and accept the candidate event at $t + \Delta t$ with probability

$$p_{\text{acc}} = \frac{\sum_{c \in \mathcal{C}} \lambda_c(t + \Delta t; \theta^* + \Delta)}{\bar{\lambda}}. \quad (12)$$

Once accepted, its event type is drawn from the categorical distribution over the normalized intensities, i.e., $c \sim \text{Categorical}(\{P(c \mid t + \Delta t, \mathcal{H}_{t+\Delta t}; \theta^* + \Delta)\}_{c \in \mathcal{C}})$ in (10).

Remark. Sequence-level conditions are commonly available in practice. For example, dates, weather, and public affairs may significantly impact daily taxi demand, so we can treat the taxi demand sequences of each day as a cluster. Given the daily news and weather broadcast (e.g., “a rainy weekday with a school bus driver strike”), we obtain the condition embedding by passing the textual description through a pretrained language model (Qwen (Zhang et al. 2025)). Alternatively, we can aggregate event-level features as condition embeddings. For user purchase sequences in e-commerce datasets, we compute statistics such as the average inter-event time, the event type distribution, and the hourly activity pattern, and then expand these features via kernel functions to characterize each user’s behavior.

Experiments

We evaluate GA-TPP on three real-world event-sequence datasets spanning distinct domains. Our experiments address three questions: (i) Does generative adaptation improve event prediction over competing conditioning and adaptation strategies? (ii) Does it generalize to out-of-distribution (OOD) conditions unseen during training? (iii) Do the adapted models simulate event sequences whose dynamics are consistent with the target conditions? All experiments are run on two NVIDIA RTX 4090 GPUs, and we report the mean and standard deviation over three runs.

Dataset	#Seq.			#Grp.	#Type	Avg. Len	d_e
	Train	Val	Test				
Taxi	185.1K	23.1K	23.1K	962	44	40.4	820
Amazon	116.1K	24.9K	24.9K	175	25	56.9	801
Citibike	105.2K	13.2K	13.2K	132	46	83.1	822

*Condition sources: Taxi/Citibike use weather (T+N) and calendar (N); Amazon uses user profile (T) and cohort (N).

Table 1: The Statistics of Datasets.*

Implementation Details

Datasets Table 1 summarizes the statistics of the three datasets, including the condition-embedding dimension d_e and the modality of each condition source (textual, numerical, or both, as noted below the table). Each dataset follows the same construction pipeline: raw records are converted into event sequences with dataset-specific event types, grouped into condition groups by their side information, and each group is associated with a condition embedding e_m derived from that side information. For **out-of-distribution (OOD)** evaluation, we hold out 20% of the condition groups entirely—none of their sequences

Method	Model	Taxi			Amazon			Citibike		
		ELL	ACC	AUC	ELL	ACC	AUC	ELL	ACC	AUC
In-Distribution (ID)										
Cond. Input	TransFeat	-2.545 $\pm$ 0.079	0.294 $\pm$ 0.004	0.827 $\pm$ 0.012	-1.038 $\pm$ 0.005	0.218 $\pm$ 0.014	0.577 $\pm$ 0.003	0.980 $\pm$ 0.008	0.405 $\pm$ 0.152	0.783 $\pm$ 0.191
Meta Learn.	MetaTPP	-2.780 $\pm$ 0.102	0.327 $\pm$ 0.016	0.762 $\pm$ 0.019	0.619 $\pm$ 0.025	0.299 $\pm$ 0.001	0.611 $\pm$ 0.001	1.325 $\pm$ 0.007	0.551 $\pm$ 0.004	0.926 $\pm$ 0.003
Uncond.	NHP	-1.762 $\pm$ 0.002	0.356 $\pm$ 0.002	0.866 $\pm$ 0.003	1.584 $\pm$ 0.544	0.264 $\pm$ 0.014	0.607 $\pm$ 0.004	1.545 $\pm$ 0.003	0.556 $\pm$ 0.001	0.923 $\pm$ 0.000
	SAHP	-2.555 $\pm$ 0.008	0.271 $\pm$ 0.010	0.844 $\pm$ 0.004	-0.681 $\pm$ 0.159	0.259 $\pm$ 0.013	0.573 $\pm$ 0.008	1.200 $\pm$ 0.004	0.557 $\pm$ 0.000	0.923 $\pm$ 0.006
	THP	-3.495 $\pm$ 0.081	0.327 $\pm$ 0.010	0.851 $\pm$ 0.009	-0.793 $\pm$ 0.042	0.315 $\pm$ 0.001	0.643 $\pm$ 0.000	0.162 $\pm$ 0.401	0.526 $\pm$ 0.011	0.923 $\pm$ 0.003
	AttNHP	-2.630 $\pm$ 0.287	0.186 $\pm$ 0.027	0.686 $\pm$ 0.030	-0.513 $\pm$ 0.006	0.294 $\pm$ 0.002	0.611 $\pm$ 0.001	-0.546 $\pm$ 0.257	0.379 $\pm$ 0.019	0.814 $\pm$ 0.004
GA-TPP(Ours)	NHP	-1.363 $\pm$ 0.007	0.358 $\pm$ 0.003	0.869 $\pm$ 0.000	1.687 $\pm$ 0.538	0.297 $\pm$ 0.017	0.619 $\pm$ 0.001	1.800 $\pm$ 0.005	0.564 $\pm$ 0.000	0.930 $\pm$ 0.000
	SAHP	-2.229 $\pm$ 0.022	0.294 $\pm$ 0.001	0.857 $\pm$ 0.009	-0.422 $\pm$ 0.214	0.290 $\pm$ 0.009	0.588 $\pm$ 0.009	1.348 $\pm$ 0.010	0.564 $\pm$ 0.000	0.930 $\pm$ 0.006
	THP	-2.164 $\pm$ 0.030	0.356 $\pm$ 0.008	0.870 $\pm$ 0.005	-0.485 $\pm$ 0.003	0.347 $\pm$ 0.000	0.646 $\pm$ 0.001	1.258 $\pm$ 0.012	0.563 $\pm$ 0.000	0.940 $\pm$ 0.001
	AttNHP	-1.620 $\pm$ 0.030	0.199 $\pm$ 0.029	0.697 $\pm$ 0.039	0.114 $\pm$ 0.017	0.340 $\pm$ 0.002	0.628 $\pm$ 0.000	0.575 $\pm$ 0.019	0.391 $\pm$ 0.014	0.830 $\pm$ 0.006
Out-of-Distribution (OOD)										
Cond. Input	TransFeat	-2.587 $\pm$ 0.080	0.289 $\pm$ 0.004	0.821 $\pm$ 0.012	-1.062 $\pm$ 0.002	0.200 $\pm$ 0.005	0.573 $\pm$ 0.003	1.108 $\pm$ 0.004	0.409 $\pm$ 0.164	0.789 $\pm$ 0.186
Meta Learn.	MetaTPP	-2.767 $\pm$ 0.107	0.323 $\pm$ 0.015	0.758 $\pm$ 0.019	0.675 $\pm$ 0.048	0.302 $\pm$ 0.006	0.612 $\pm$ 0.002	1.415 $\pm$ 0.012	0.558 $\pm$ 0.001	0.925 $\pm$ 0.003
Uncond.	NHP	-1.786 $\pm$ 0.005	0.355 $\pm$ 0.002	0.866 $\pm$ 0.002	1.604 $\pm$ 0.551	0.264 $\pm$ 0.014	0.615 $\pm$ 0.001	1.592 $\pm$ 0.005	0.558 $\pm$ 0.001	0.924 $\pm$ 0.000
	SAHP	-2.614 $\pm$ 0.006	0.268 $\pm$ 0.010	0.836 $\pm$ 0.004	-0.802 $\pm$ 0.162	0.238 $\pm$ 0.010	0.575 $\pm$ 0.010	1.304 $\pm$ 0.005	0.561 $\pm$ 0.000	0.924 $\pm$ 0.006
	THP	-3.569 $\pm$ 0.073	0.327 $\pm$ 0.013	0.847 $\pm$ 0.008	-0.780 $\pm$ 0.045	0.335 $\pm$ 0.000	0.643 $\pm$ 0.002	0.570 $\pm$ 0.231	0.537 $\pm$ 0.010	0.926 $\pm$ 0.002
	AttNHP	-2.709 $\pm$ 0.277	0.185 $\pm$ 0.027	0.689 $\pm$ 0.036	-0.462 $\pm$ 0.007	0.286 $\pm$ 0.002	0.624 $\pm$ 0.000	-0.039 $\pm$ 0.215	0.396 $\pm$ 0.019	0.820 $\pm$ 0.005
GA-TPP(Ours)	NHP	-1.382 $\pm$ 0.015	0.361 $\pm$ 0.009	0.871 $\pm$ 0.005	1.727 $\pm$ 0.549	0.287 $\pm$ 0.021	0.622 $\pm$ 0.005	1.732 $\pm$ 0.003	0.567 $\pm$ 0.000	0.931 $\pm$ 0.000
	SAHP	-2.318 $\pm$ 0.014	0.291 $\pm$ 0.001	0.857 $\pm$ 0.002	-0.615 $\pm$ 0.203	0.247 $\pm$ 0.011	0.594 $\pm$ 0.011	1.387 $\pm$ 0.005	0.567 $\pm$ 0.002	0.930 $\pm$ 0.007
	THP	-2.189 $\pm$ 0.055	0.351 $\pm$ 0.008	0.862 $\pm$ 0.005	-0.544 $\pm$ 0.002	0.352 $\pm$ 0.001	0.652 $\pm$ 0.001	1.043 $\pm$ 0.096	0.556 $\pm$ 0.002	0.936 $\pm$ 0.001
	AttNHP	-1.710 $\pm$ 0.011	0.203 $\pm$ 0.034	0.695 $\pm$ 0.032	0.025 $\pm$ 0.012	0.319 $\pm$ 0.001	0.640 $\pm$ 0.002	0.209 $\pm$ 0.251	0.402 $\pm$ 0.019	0.832 $\pm$ 0.008

Table 2: ID and OOD performance comparison. Subscripts denote standard deviations over three runs; **bold** marks the better of each backbone and its GA-TPP variant.

or statistics is used in any training stage, i.e., a zero-shot, group-level protocol without any support set; the sequences of the remaining groups are split 80%/10%/10% into train/validation/test sets, and the test sequences serve for **in-distribution (ID)** evaluation.

- **Taxi** (Whong 2014): NYC taxi records (2010–2013). The city is partitioned into 22 zones by k -means (selected via the Silhouette coefficient), and each zone is paired with a pick-up/drop-off role, yielding 44 event types. Sequences are grouped by calendar date (962 groups), with conditions built from daily and hourly weather (TimeAndDate.com 2026) and calendar attributes.
- **Amazon** (Hou et al. 2024): Amazon Reviews 2023, retaining the most active users over 25 product categories as event types. Users are grouped into 175 cohorts by k -means on their category preferences, with conditions combining a cohort-level Qwen embedding (Zhang et al. 2025) and an 8-dimensional vector of review and temporal features.
- **Citibike** (NYCBike 2014): NYC Citi Bike trips (2014–2016), organized as weekly per-bicycle trajectories. Station locations are clustered by k -means and paired with a start/end role, yielding 46 event types. Sequences

are grouped by ISO week, with conditions built from weather, spatio-temporal, and calendar information.

Backbones To examine the compatibility, we apply it to four representative neural TPP architectures: **NHP** (Mei and Eisner 2017) (continuous-time LSTM), **SAHP** (Zhang et al. 2020) (self-attentive Hawkes process), **THP** (Zuo et al. 2020) (Transformer Hawkes process), and **AttNHP** (Yang, Mei, and Eisner 2021) (attentive neural Hawkes process).

Baselines We compare four methods, categorized by how they incorporate conditions, each evaluated on top of every backbone: (1) **Uncond.**: the backbone trained unconditionally on all sequences, ignoring conditions; (2) **TransFeat-TPP** (Meng et al. 2024): an input-level conditioning approach that injects covariate embeddings into the event representations; (3) **MetaTPP** (Bae et al. 2023): a meta-learning approach that performs gradient-based adaptation across event-sequence tasks; (4) **GA-TPP** (ours): a diffusion model generates a condition-specific adapter from e_m , requiring no per-condition gradient updates and thus directly applying to unseen (OOD) conditions. Among these, Uncond. uses no conditions, TransFeat-TPP applies input-level conditioning,

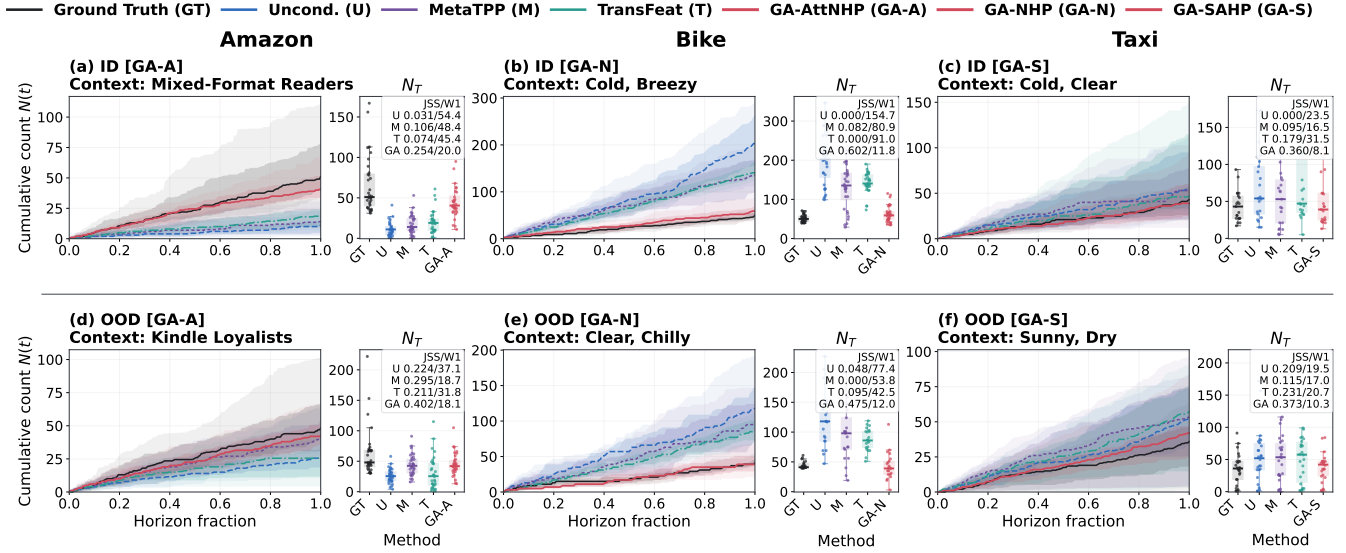


Figure 2: Simulated counting processes $N(t)$ and terminal-count N_T distributions under representative ID (top) and OOD (bottom) condition groups. Annotations report the JS similarity ($\uparrow$) and W_1 distance ($\downarrow$) to the ground truth.

while MetaTPP and GA-TPP perform parameter-level adaptation.

Evaluation Measurements We evaluate two tasks. *i*) For **event prediction**, we report the log-likelihood per event (ELL), type prediction accuracy (ACC), and area under the ROC curve (AUC), measuring temporal density estimation, one-step type prediction, and ranking quality, respectively. *ii*) For **sequence simulation**, we compare the terminal event counts N_T of simulated and ground-truth sequences within each condition group, reporting the **Jensen-Shannon similarity (JSS)** (higher is better) and the **1-Wasserstein distance (W_1)** (lower is better) between two N_T distributions.

Comparison Experiments

Table 2 reports both the ID and OOD results across all three datasets. First, GA-TPP improves every backbone on almost all dataset-metric pairs in the ID setting, often by large margins on ELL. Second, the gains persist in the zero-shot OOD setting without any gradient update, and the ID-OOD gap of GA-TPP remains small, indicating that the diffusion prior transfers adaptation to unseen conditions rather than memorizing training clusters. Third, GA-TPP consistently outperforms both input-level conditioning (TransFeat-TPP) and online meta-adaptation (MetaTPP), validating parameter-level generative adaptation driven purely by side information.

Simulation Experiments

To answer question (iii), we simulate complete event sequences under both ID and OOD conditions on all three datasets. For each selected condition group, every method simulates event sequences over the same horizon as the

ground-truth data via Ogata’s thinning algorithm, and we examine the per-sequence cumulative counting processes $N(t)$ as well as the marginal distribution of the terminal counts N_T . Figure 2 presents representative groups: for each dataset, one ID and one OOD condition (e.g., “Cold, Breezy” weather for Citibike and the “Kindle Loyalists” cohort for Amazon), comparing the unconditional backbone, MetaTPP, TransFeat-TPP, and GA-TPP against the ground truth. Two patterns emerge. First, the mean counting processes of GA-TPP track the empirical curves closely in both settings, whereas the unconditional and input-level baselines systematically over- or under-estimate the event rates, especially under OOD conditions where the target dynamics deviate from the training average. Second, the N_T strip plots show that GA-TPP produces terminal-count distributions well aligned with the ground truth, achieving the highest JS similarity and the lowest W_1 distance in all six panels. These results demonstrate that generative adaptation yields simulators whose stochastic dynamics, not merely their point predictions, are consistent with the target conditions.

Ablation Studies

Diffusion vs. a Deterministic Generator We first test whether a diffusion generator is necessary, or whether a simpler deterministic map suffices, by replacing the diffusion model with a Multi-Layer Perceptron (MLP) that regresses directly from the condition embedding e_m to LoRA parameters. Table 3 compares the two generators on the Taxi OOD groups across all four backbones. The diffusion generator wins on every backbone-metric pair, and the advantage is most pronounced on ELL, the metric most sensitive to how well the adapted intensity fits the target dynamics; the gaps on ACC/AUC are smaller, es-

pecially for the RNN-based NHP whose type prediction is nearly saturated. This supports our design: mapping a condition to adapters is inherently one-to-many, and a deterministic regressor tends to average over plausible adapters, whereas the diffusion prior models the full adapter distribution.

Generator	Metric	AttNHP	SAHP	NHP	THP
MLP	ELL $\uparrow$	-2.091 (0.021)	-2.433 (0.055)	-1.522 (0.015)	-2.793 (0.065)
	ACC $\uparrow$	0.191 (0.038)	0.273 (0.006)	0.350 (0.002)	0.339 (0.010)
	AUC $\uparrow$	0.690 (0.036)	0.844 (0.005)	0.862 (0.001)	0.853 (0.006)
Diffusion	ELL $\uparrow$	-1.710 (0.011)	-2.318 (0.014)	-1.382 (0.015)	-2.189 (0.055)
	ACC $\uparrow$	0.203 (0.034)	0.291 (0.001)	0.361 (0.009)	0.351 (0.008)
	AUC $\uparrow$	0.695 (0.032)	0.857 (0.002)	0.871 (0.005)	0.862 (0.005)

Table 3: Adapter generator ablation on Taxi OOD groups. Parentheses show standard deviations over three runs.

Which Weight Matrices to Adapt? On Taxi with THP, we inject LoRAs into different subsets of the projection matrices $\{\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v, \mathbf{W}_o\}$ and vary the number of adapted layers, adjusting the rank so that the total trainable parameters remain constant (Table 4). Under the same budget, jointly adapting $\mathbf{W}_q$ and $\mathbf{W}_v$ performs best in both settings, echoing Hu et al. (2021): spreading the budget over query and value projections beats concentrating it on a single matrix, while further including $\mathbf{W}_k$ or $\mathbf{W}_o$ dilutes the per-matrix rank without gains. Adapting all layers also outperforms adapting only the first or last layer, suggesting that condition-specific variations affect event representations at multiple depths; we therefore adapt $\{\mathbf{W}_q, \mathbf{W}_v\}$ of all layers by default.

Weights	Layers	ID		OOD	
		ELL $\uparrow$	ACC $\uparrow$	ELL $\uparrow$	ACC $\uparrow$
$\mathbf{W}_v$	all	-2.192	0.347	-2.301	0.342
$\mathbf{W}_q$	all	-2.195	0.347	-2.283	0.341
$\mathbf{W}_k$	all	-2.213	0.342	-2.301	0.337
$\mathbf{W}_q, \mathbf{W}_v$	all	-2.164	0.356	2.189	0.351
$\mathbf{W}_q, \mathbf{W}_k$	all	-2.198	0.345	-2.278	0.340
$\mathbf{W}_k, \mathbf{W}_v$	all	-2.191	0.348	-2.274	0.343
$\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v$	all	-2.179	0.350	-2.260	0.345
$\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v, \mathbf{W}_o$	all	-2.184	0.348	-2.265	0.343
All layers	all	-2.164	0.356	2.189	0.351
Last layer	last	-2.201	0.346	-2.285	0.341
First layer	first	-2.209	0.344	-2.293	0.339

Table 4: Ablation on LoRA injection locations on Taxi with THP, a fixed parameter budget: attention projections (top) and adapted layers (bottom).

Sensitivity to Grouping Granularity In Table 5, we study the effect of grouping granularity by merging the original Taxi groups at different ratios, where smaller ratios yield fewer but larger groups. Since merging alters the ID/OOD partition, we report the gain of GA-TPP over Uncond., which shares identical splits at each ratio. GA-TPP improves consistently at all granularities; the ELL gain grows as groups become finer—finer groups exhibit more distinctive dynamics, making adaptation more valuable—while the stable ACC/AUC gains show that GA-TPP requires no careful tuning of the granularity.

GA-TPP Gain	Grouping Ratio				
	1/2	1/4	1/8	1/16	1/32
Δ ELL $\uparrow$	+1.214	+1.132	+0.984	+0.826	+0.668
Δ ACC $\uparrow$	+0.018	+0.022	+0.023	+0.022	+0.022
Δ AUC $\uparrow$	+0.014	+0.016	+0.017	+0.017	+0.016

Table 5: Performance gains of GA-TPP over Uncond. on Taxi with THP under the ID setting. A ratio of $1/k$ denotes the fraction of groups retained relative to the original grouping. Values report GA-TPP minus Uncond.

Efficiency Analysis We compare the per-condition adaptation cost of GA-TPP against gradient-based LoRA retraining on Taxi with 962 condition groups. Retraining a LoRA adapter takes 56.82s per condition on average, i.e., 15.18 GPU-hours for all groups. In contrast, GA-TPP draws 16 adapter samples (0.637s per draw) and averages them into the final adapter, taking 10.20s per condition end-to-end—a $5.57\times$ speedup. Moreover, the diffusion generator is trained once offline and amortized across all conditions, and, requiring no event data from the target condition, GA-TPP is the only applicable option in zero-shot OOD scenarios.

Conclusion and Future Work

In this study, we have proposed GA-TPP, which learns a conditional diffusion model over LoRA parameters and injects the generated adapters into a pretrained TPP, thereby adapting to unseen conditions from side information alone, without per-condition data or gradient updates. Experiments on three real-world datasets and four TPP backbones show consistent improvements in both event prediction and sequence simulation, especially under zero-shot out-of-distribution conditions, establishing parameter-level generative control as a practical paradigm for non-stationary event dynamics.

Limitations and Future Work. Our method relies on a predefined grouping of event sequences and trains one LoRA per group, whose quality and number affect both performance and training cost; we plan to learn the grouping structure jointly with the diffusion model, and to accelerate adapter generation with more efficient generators, e.g., flow-based models with ten or fewer sampling steps (Liu, Gong et al. 2023; Geng et al. 2025).

References

- Bae, W.; Ahmed, M. O.; Tung, F.; and Oliveira, G. L. 2023. Meta temporal point processes. *arXiv preprint arXiv:2301.12023*.
- Dong, Z.; Fan, Z.; and Zhu, S. 2025. Conditional generative modeling for high-dimensional marked temporal point processes. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, 248–259.
- Du, N.; Dai, H.; Trivedi, R.; Upadhyay, U.; Gomez-Rodriguez, M.; and Song, L. 2016. Recurrent marked temporal point processes: Embedding event history to vector. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, 1555–1564.
- Geng, Z.; Deng, M.; Bai, X.; Kolter, J. Z.; and He, K. 2025. Mean flows for one-step generative modeling. *arXiv preprint arXiv:2505.13447*.
- Ha, D.; Dai, A.; and Le, Q. V. 2016. Hypernetworks. *arXiv preprint arXiv:1609.09106*.
- Hawkes, A. G. 1971. Spectra of some self-exciting and mutually exciting point processes. *Biometrika*, 58(1): 83–90.
- Ho, J.; Jain, A.; and Abbeel, P. 2020. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33: 6840–6851.
- Hou, Y.; Li, J.; He, Z.; Yan, A.; Chen, X.; and McAuley, J. 2024. Bridging Language and Items for Retrieval and Recommendation. *arXiv preprint arXiv:2403.03952*.
- Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.
- Kalair, K.; Connaughton, C.; and Alaimo Di Loro, P. 2021. A non-parametric Hawkes process model of primary and secondary accidents on a UK smart motorway. *Journal of the Royal Statistical Society Series C: Applied Statistics*, 70(1): 80–97.
- Kingman, J. F. C. 1992. *Poisson processes*, volume 3. Clarendon Press.
- Kong, Q.; Zhang, Y.; Liu, Y.; Tong, P.; Liu, E.; and Zhou, F. 2026. Byte-token enhanced language models for temporal point processes analysis. In *Proceedings of the ACM Web Conference 2026*, 7013–7023.
- Li, S.; Xiao, S.; Zhu, S.; Du, N.; Xie, Y.; and Song, L. 2018. Learning temporal point processes via reinforcement learning. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, 10804–10814.
- Liu, X.; Gong, C.; et al. 2023. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow. In *The Eleventh International Conference on Learning Representations*.
- Liu, Z.; and Quan, Y. 2024. TPP-LLM: Modeling Temporal Point Processes by Efficiently Fine-Tuning Large Language Models. *arXiv preprint arXiv:2410.02062*.
- Mei, H.; and Eisner, J. M. 2017. The neural Hawkes process: A neurally self-modulating multivariate point process. *Advances in neural information processing systems*, 30.
- Mei, H.; Wan, T.; and Eisner, J. 2020. Noise-contrastive estimation for multivariate point processes. *Advances in neural information processing systems*, 33: 5204–5214.
- Meng, Z.; Li, B.; Fan, X.; Li, Z.; Wang, Y.; Chen, F.; and Zhou, F. 2024. Transfeat-tp: An interpretable deep covariate temporal point processes. *arXiv preprint arXiv:2407.16161*.
- Mou, C.; Wang, X.; Xie, L.; Wu, Y.; Zhang, J.; Qi, Z.; and Shan, Y. 2024. T2I-adapter: Learning adapters to dig out more controllable ability for text-to-image diffusion models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 4296–4304.
- NYCBike. 2014. Citi Bike Trip Histories. <https://s3.amazonaws.com/tripdata/index.html>. Accessed: 2024-01-15.
- Okawa, M.; Iwata, T.; Kurashima, T.; Tanaka, Y.; Toda, H.; and Ueda, N. 2019. Deep mixture point processes: Spatio-temporal event prediction with rich contextual information. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 373–383.
- Shchur, O.; Türkmen, A. C.; Januschowski, T.; and Günnemann, S. 2021. Neural temporal point processes: A review. *arXiv preprint arXiv:2104.03528*.
- Shen, C.; Zhao, J.; Zhang, X.; Yu, W.; He, M.; and Fan, J. 2025. Paragon: Parameter Generation for Controllable Multi-Task Recommendation. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*, 370–380.
- Song, Y.; Sohl-Dickstein, J.; Kingma, D. P.; Kumar, A.; Ermon, S.; and Poole, B. 2020. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*.
- Sun, Z.; Si, Z.; Zhang, X.; Zang, X.; Song, Y.; Xu, H.; and Xu, J. 2024. To Search or to Recommend: Predicting Open-App Motivation with Neural Hawkes Process. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1018–1028.
- TimeAndDate.com. 2026. New York City Historical Weather. <https://www.timeanddate.com/weather/usa/new-york/historic>. Accessed: 2026-02-01.
- Vaswani, A. 2017. Attention is all you need. *Advances in Neural Information Processing Systems*.
- Wang, Q.; Cheng, M.; Yuan, S.; and Xu, H. 2023. Hierarchical contrastive learning for temporal point processes. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, 10166–10174.
- Whong, C. 2014. FOILing NYC’s Taxi Trip Data. https://chriswhong.com/open-data/foil_nyc_taxi/. Accessed: 2026-02-01.

Xiao, S.; Farajtabar, M.; Ye, X.; Yan, J.; Song, L.; and Zha, H. 2017. Wasserstein learning of deep generative point process models. *Advances in neural information processing systems*, 30.

Xu, H.; Luo, D.; and Zha, H. 2017. Learning hawkes processes from short doubly-censored event sequences. In *International Conference on Machine Learning*, 3831–3840. PMLR.

Xu, H.; Wu, W.; Nemati, S.; and Zha, H. 2016. Patient flow prediction via discriminative learning of mutually-correcting processes. *IEEE transactions on Knowledge and Data Engineering*, 29(1): 157–171.

Xue, S.; Shi, X.; Chu, Z.; Wang, Y.; Zhou, F.; Hao, H.; Jiang, C.; Pan, C.; Xu, Y.; Zhang, J. Y.; et al. 2023. Easytpp: Towards open benchmarking the temporal point processes. *arXiv preprint arXiv:2307.08097*.

Yang, C.; Mei, H.; and Eisner, J. 2021. Transformer embeddings of irregularly spaced events and their participants. *arXiv preprint arXiv:2201.00044*.

Yuan, Y.; Ding, J.; Shao, C.; Jin, D.; and Li, Y. 2023. Spatio-temporal diffusion point processes. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 3173–3184.

Zhang, L.; Rao, A.; and Agrawala, M. 2023. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 3836–3847.

Zhang, Q.; Lipani, A.; Kirnap, O.; and Yilmaz, E. 2020. Self-attentive Hawkes process. In *International conference on machine learning*, 11183–11193. PMLR.

Zhang, Q.; Lipani, A.; and Yilmaz, E. 2021. Learning Neural Point Processes with Latent Graphs. In *Proceedings of the Web Conference 2021*, 1495–1505.

Zhang, Y.; Li, M.; Long, D.; Zhang, X.; Lin, H.; Yang, B.; Xie, P.; Yang, A.; Liu, D.; Lin, J.; et al. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*.

Zhou, K.; Zha, H.; and Song, L. 2013. Learning social infectivity in sparse low-rank networks using multi-dimensional hawkes processes. In *Artificial Intelligence and Statistics*, 641–649. PMLR.

Zuo, S.; Jiang, H.; Li, Z.; Zhao, T.; and Zha, H. 2020. Transformer hawkes process. In *International conference on machine learning*, 11692–11702. PMLR.